
Machine Learning Lab Report

Seven Interactive Real-Life Problem Demonstrations

Course code: CSE 816
Student name: [Write Name Here]
Student ID: 20701003
Report type: Complete laboratory report
Prepared on: September 13, 2026

Department of Computer Science and Engineering

This report documents the Perceptron, K-Nearest Neighbors, Decision Tree, Support Vector Machine, K-Means, Hidden Markov Model, and CNN/YOLO projects in the supplied CSE 816 workspace. All medical examples are synthetic teaching demonstrations and are not diagnostic systems.

Declaration and Scope

This document was prepared from the current source code, datasets, tests, `README.md` files, and project reports in the workspace. The report describes the actual implementation rather than an imagined library version. The fixed header shows CSE 816 on the left and 20701003 on the right on every page; every footer contains only the centered page number.

The applications are educational. The BMI, diabetes, and cardiac-triage projects must not be used to make health decisions. The customer dataset is fictional, the Perceptron student points are user-created teaching examples, the HMM coins are a probability puzzle, and uploaded CNN images are inference inputs rather than new training data.

Algorithms and problems covered

Algorithm	Problem title	Architecture
Perceptron	Predicting a synthetic student Pass/Fail label from study time and attendance	Flask API trains; browser animates
KNN	Sorting an adult BMI using the five nearest saved examples	Python exports; browser predicts
Decision Tree	Following readable yes/no rules for synthetic cardiac-triage cards	Python trains; browser predicts
Linear SVM	Visualizing a wide separating margin between two diabetes-screening example groups	Python trains; browser predicts
K-Means	Grouping fictional customers by age and shopping score	Browser trains and predicts
HMM	Guessing which hidden coin generated a visible H/T sequence	Browser decodes locally
CNN/YOLO	Finding and drawing boxes around everyday objects in a photo	Flask runs pretrained inference

Contents

Declaration and Scope	2
1 Perceptron	9
1.1 Problem Description	9
1.2 Algorithm Description	9
1.2.1 The chalk-line explanation	9
1.2.2 A numerical step	10
1.2.3 Core code and data flow	10
1.3 Dataset Description	11
1.4 Training	12
1.4.1 How to run and train	12
1.4.2 Parameters a learner may change	13
1.5 Website Design and Answer Flow	13
1.6 Comprehensive User Manual	14
1.6.1 Screen guide	14
1.6.2 Every input and control	15
1.6.3 How to perform a successful experiment	16
1.6.4 How to read every output	16
1.6.5 Troubleshooting	16
1.7 Conclusion	17
2 K-Nearest Neighbors (KNN)	18
2.1 Problem Description	18
2.2 Algorithm Description	18
2.2.1 The neighbor-vote story	18
2.3 Dataset Description	19

2.4	Training	20
2.5	Website Design and Answer Flow	21
2.6	Comprehensive User Manual	22
2.6.1	Inputs and buttons	22
2.6.2	Outputs	22
2.6.3	Worked example	23
2.6.4	Troubleshooting	23
2.7	Conclusion	23
3	Decision Tree	24
3.1	Problem Description	24
3.2	The Algorithm	24
3.2.1	A tree made from questions	24
3.2.2	How the program chooses a question	25
3.3	Dataset Description	26
3.4	Training	27
3.5	Website	27
3.6	Comprehensive User Manual	28
3.6.1	Start and enter a card	28
3.6.2	Read every output	28
3.7	Conclusion	29
4	Support Vector Machine	30
4.1	Problem Description	30
4.2	The Algorithm	30
4.2.1	A wide road between two groups	30
4.3	Dataset Description	31
4.4	Training	32
4.5	Website	32
4.6	Comprehensive User Manual	33
4.7	Conclusion	34
5	K-Means Clustering	35
5.1	Problem Description	35
5.2	The Algorithm	35

5.2.1	Move two team flags until nobody changes team	35
5.3	Dataset Description	37
5.4	Training	37
5.5	Website	38
5.6	Comprehensive User Manual	39
5.7	Conclusion	39
6	Hidden Markov Model	40
6.1	Problem Description	40
6.2	The Algorithm	40
6.2.1	Model pieces	40
6.2.2	Viterbi: the single best hidden route	41
6.2.3	Forward-backward: belief at each position	41
6.3	Dataset Description	42
6.4	Training and Parameter Experimentation	43
6.5	Website	43
6.6	Comprehensive User Manual	44
6.7	Conclusion	45
7	CNN and YOLO Object Detection	46
7.1	Problem Description	46
7.2	The Algorithm	46
7.2.1	From pixels to visual clues	46
7.2.2	YOLO detection in one model pass	46
7.3	Dataset Description	48
7.3.1	What data the current application uses	48
7.4	Training and Model Use	49
7.4.1	Run the supplied pretrained detector	49
7.4.2	Optional custom training	49
7.5	Website	50
7.6	Comprehensive User Manual	51
7.6.1	Choose an input	51
7.6.2	Run and understand the answer	51
7.7	Conclusion	52

8 Overall Conclusion	53
A Verification and Reproduction	54
A.1 Commands verified in this workspace	54
A.2 One-page run checklist	55
B Glossary for a First-Time Reader	56
C LaTeX Build and Error-Fixing Guide	57
C.1 Only required personal edit	57
C.2 Build commands	57
C.3 Common errors and ready fixes	58
References	59

List of Figures

1.1	Perceptron website request and response flow.	14
1.2	Student Perceptron Lab website, captured from the running local app. . .	15
2.1	KNN training-to-browser flow.	21
2.2	BMI Neighbor website with the two measurement inputs and result panel.	22
3.1	The exact tree exported by the current training script.	25
3.2	Decision-tree training and browser prediction flow.	27
3.3	Live decision-tree page showing a HIGH route for BP 155 and HR 130. . .	28
4.1	SVM training, export, and browser scoring flow.	33
4.2	Live SVM page with the higher preset (150 mg/dL, BMI 33).	33
5.1	K-Means page-load training and user classification flow.	38
5.2	Live customer explorer after classifying age 48 and score 86.	38
6.1	Hidden Coin Detective inference flow.	43
6.2	Live HMM page decoding the heads-heavy preset.	44
7.1	End-to-end Sightline request flow.	50
7.2	Live Sightline upload interface before a photograph is selected.	51

List of Tables

1.1	Perceptron implementation map.	11
1.2	Perceptron dataset fields.	12
1.3	Perceptron tuning guide.	13
1.5	Perceptron outputs in plain language.	16
2.1	KNN implementation map.	19
2.2	KNN CSV columns and verified contents.	20
2.3	KNN parameters and their effects.	21
2.4	KNN result meanings.	23
3.1	Decision-tree functions and data flow.	26
3.2	Meaning of each cardiac demonstration column.	26
4.1	Linear-SVM functions and passed values.	31
4.2	Meaning of the SVM dataset columns.	32
5.1	K-Means browser functions.	36
5.2	Fields in each fictional customer object.	37
6.1	HMM functions, inputs, and outputs.	42
6.2	HMM input data and model controls.	42
7.1	CNN web application functions and information flow.	48
7.2	Input image information and its meaning.	49
C.1	Common LaTeX problems and safe corrections.	58

Lab 1

Perceptron

Problem: Student Exam Pass/Fail Prediction

1.1 Problem Description

The first website turns a supervised binary-classification lesson into a visible classroom map. Each synthetic student has two inputs: study hours per week, $p_1 \in [0, 10]$, and normalized attendance, $p_2 \in [0, 10]$. An attendance value of 7.5 represents 75%. The target t is 1 for Pass or 0 for Fail. The goal is to learn one straight line that places the green Pass examples on one side and the red Fail examples on the other.

This problem is suitable for a single neuron only when the two classes are linearly separable. The XOR preset deliberately violates that condition. It shows the model's limitation: no amount of repeating the same straight-line rule can separate labels placed at opposite corners. The design and update rule follow Chapter 4 of Hagan et al. [1], while the Perceptron originates in Rosenblatt's early work on trainable threshold systems [2].

Safety boundary: The points are invented and the result is not a grading, admissions, or student-risk decision. A real educational model would require representative data, fairness analysis, uncertainty measurement, governance, and human review.

1.2 Algorithm Description

1.2.1 The chalk-line explanation

Imagine placing green and red cards on a floor. A robot has one piece of chalk. It guesses which side each card belongs on. When it is wrong, it turns or moves the chalk line; when it is right, it does nothing. One full visit to every card is an *epoch*. The learned line is the decision boundary.

The input vector, parameter vector, net input, and hard-limit output are

$$p = \begin{bmatrix} p_1 \\ p_2 \end{bmatrix}, \quad w = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}, \quad n = w^T p + b, \quad (1.1)$$

$$a = \text{hardlim}(n) = \begin{cases} 1, & n \geq 0, \\ 0, & n < 0. \end{cases} \quad (1.2)$$

The error is $e = t - a$. For learning rate α , one mistake changes the parameters using

$$w_{\text{new}} = w_{\text{old}} + \alpha e p, \quad b_{\text{new}} = b_{\text{old}} + \alpha e. \quad (1.3)$$

The project fixes $\alpha = 1$, matching the unified rule in Hagan et al. [1]. If $e = 1$, the sample vector is added; if $e = -1$, it is subtracted; if $e = 0$, no parameter changes.

1.2.2 A numerical step

For $p = [2, 3]^T$, $t = 0$, $w = [0, 0]^T$, and $b = 0$, the net input is zero and `hardlim(0)` returns 1. Therefore $e = 0 - 1 = -1$, so the next parameters are $w = [-2, -3]^T$ and $b = -1$. The website stores both old and new values so the animation can explain this exact correction.

1.2.3 Core code and data flow

```

1 net = float(self.weights @ sample + self.bias)
2 activation = int(self.hardlim(net))
3 error = int(t - activation)
4
5 if error != 0:
6     self.weights = self.weights + self.learning_rate * error * sample
7     self.bias = self.bias + self.learning_rate * error

```

Listing 1.1: One Perceptron update from `perceptron.py`.

Table 1.1: Perceptron implementation map.

Function	Input / return	Role and calling flow
<code>Perceptron(input_dim, learning_rate)</code>	2 and 1.0; model object	Validates parameters and creates zero weights and bias. Called by the Flask training route.
<code>hardlim(net)</code>	Scalar or array; 0/1	Applies $n \geq 0$. Called by <code>predict</code> and <code>train_step</code> .
<code>_coerce_inputs(p)</code>	Vector or matrix; array plus single/-batch flag	Checks shape and finite values before matrix multiplication.
<code>_coerce_dataset(P,T)</code>	Sample matrix and targets; aligned arrays	Checks row counts, non-empty data, and binary targets.
<code>net_input(p)</code>	One or many points; score(s)	Computes <code>inputs @ weights + bias</code> ; NumPy broadcasts b .
<code>predict(p)</code>	One or many points; class(es)	Calls <code>net_input</code> , then <code>hardlim</code> .
<code>train_step(p,t)</code>	One point and target; diagnostics dictionary	Predicts, calculates e , updates immediately, and returns JSON-safe old/new state.
<code>train_epoch(P,T)</code>	Complete set; step history	Calls <code>train_step</code> in submitted order once per row.
<code>fit(P,T,max_epochs)</code>	Data and limit; histories and final model	Repeats epochs until accuracy is 1.0 or the limit is reached.
<code>/api/train</code>	JSON points; JSON training result	Validates the request, trains a fresh model, and returns every animation snapshot.

The main data structures are a NumPy matrix P of shape $(Q, 2)$, a target vector T of length Q , a two-element weight array, a scalar bias, a list of step dictionaries, and a list of epoch summaries. Immediate updates make this online learning: sample $q + 1$ sees the parameters produced by sample q .

1.3 Dataset Description

There is no personal student database. The user makes a temporary dataset by clicking the graph or loading one of three literal JavaScript presets. This is a synthetic design choice based on the supervised input/target-pair notation in Hagan et al. [1].

Table 1.2: Perceptron dataset fields.

Field	Allowed values	Meaning
p1	0 to 10	Synthetic study hours per week; horizontal graph coordinate.
p2	0 to 10	Normalized attendance; multiply by 10 for percent.
target	0 or 1	Requested Fail or Pass class. This is the teaching answer, not a measured prediction outcome.

The Clearly Separable preset contains ten points, five per class. The Boundary Edge preset also contains ten points but places several close to a possible line. The XOR-like preset contains four opposite-corner points. The server accepts 2 to 50 points, requires both labels, rejects non-finite or out-of-range values, and rejects identical coordinates carrying conflicting targets. Data exists only in browser memory and the request; no database is written.

1.4 Training

1.4.1 How to run and train

```

1 cd C:\Users\User\machine_learning\perceptron\student_perceptron_app
2 python -m venv .env
3 .\.env\Scripts\Activate.ps1
4 python -m pip install -r requirements.txt
5 python app.py

```

Listing 1.2: Run the Perceptron website.

Open <http://localhost:5000>, add both green and red points, and press **Train (Animated)**. The browser sends `{points:[...], max_epochs:100}` to `POST /api/train`. Flask creates a fresh `Perceptron(2,1.0)`, calls `fit`, saves the final local snapshot, and returns initial state, sample-by-sample history, epoch history, convergence status, boundary metadata, and final accuracy.

For direct Python use:

```

1 from perceptron import Perceptron
2
3 P = [[1, 2], [2, 1], [8, 8], [9, 7]]
4 T = [0, 0, 1, 1]
5 result = Perceptron(input_dim=2, learning_rate=1.0).fit(
6     P, T, max_epochs=100
7 )
8 print(result["final"])

```

Listing 1.3: Train without the website.

1.4.2 Parameters a learner may change

Table 1.3: Perceptron tuning guide.

Parameter	Current	Effect of changing it
learning_rate	1.0	Smaller values move the line less per mistake; larger values move it more. For a classic separable Perceptron, changing the positive step size usually changes the route and parameter scale, not the existence of a separator.
max_epochs	100	A larger limit permits more passes but cannot make one line solve XOR. A smaller value returns sooner and may stop before a difficult separable set converges.
Point order	insertion order	Online updates depend on order, so the animation and final separating line can differ even when accuracy is the same.
input_dim	2	More features require matching input columns and lose the simple two-dimensional drawing.

The verified suite ran 12 Perceptron/API tests successfully, including exact hard-limit boundary behavior, the update equation, separable convergence, and bounded XOR non-convergence.

1.5 Website Design and Answer Flow

The browser owns the points, animation cursor, drawing, and telemetry. Flask owns request validation and the NumPy training operation. Canvas pixel coordinates use a downward y axis, while feature coordinates use an upward p_2 axis. The conversion is

$$x = x_L + \frac{p_1}{10}W, \quad y = y_B - \frac{p_2}{10}H, \quad (1.4)$$

and the inverse subtracts the canvas margins, divides by the plot size, and multiplies by 10.

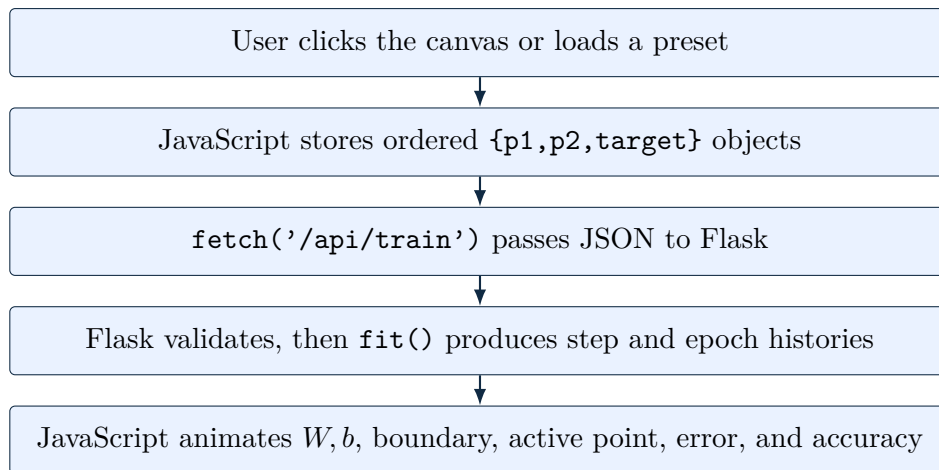


Figure 1.1: Perceptron website request and response flow.

The decision boundary is $w_1p_1 + w_2p_2 + b = 0$. The client safely draws a vertical line when w_2 is approximately zero, a slope-intercept line otherwise, and no line when both weights are zero. The lightly shaded side satisfies $w^T p + b \geq 0$.

1.6 Comprehensive User Manual

1.6.1 Screen guide

Figure 1.2 shows the starting state. The large left card is the classroom map. The right column contains dataset controls, training buttons, speed, and live mathematics.

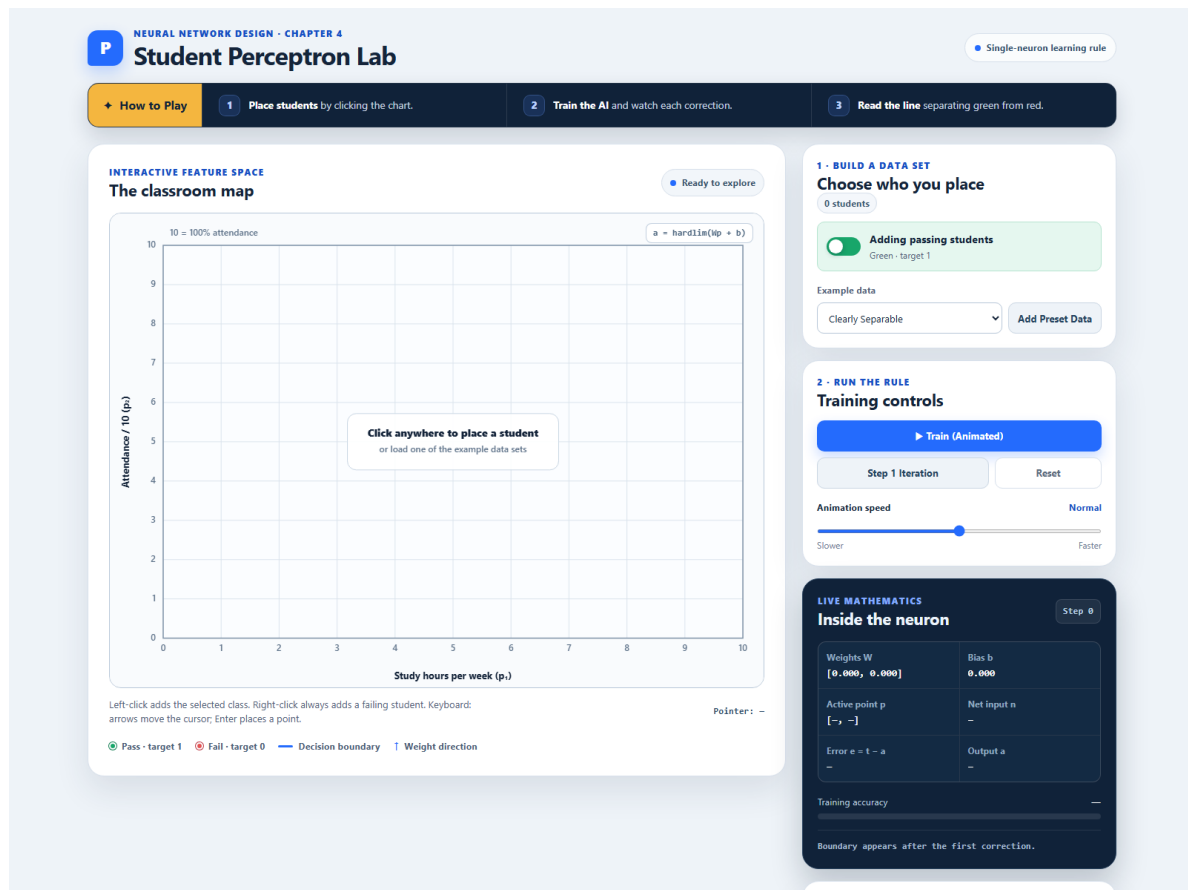


Figure 1.2: Student Perceptron Lab website, captured from the running local app.

1.6.2 Every input and control

Control	What the user does and what it means
Class switch	Green means new clicks receive target 1 (Pass); red means target 0 (Fail). A right-click always adds Fail.
Classroom map	Horizontal position is study hours; vertical position is attendance divided by 10. Click inside the numbered square only.
Example data list	Choose Clearly Separable, Boundary Edge Case, or XOR-like, then press Add Preset Data . Presets are added to any existing points if space remains.
Train (Animated)	Requests the complete training history, resets to the zero model, and plays every stored sample visit.
Step 1 Iteration	Requests the same history if needed, then advances exactly one sample.
Animation speed	Changes playback speed only; it does not change learning mathematics.

Reset	Cancels animation, deletes temporary points, restores zero parameters, and returns placement to Pass.
Keyboard operation	Focus the graph, use arrow keys to move the cursor, and press Enter or Space to place a point.

1.6.3 How to perform a successful experiment

1. Choose **Clearly Separable** and press **Add Preset Data**.
2. Confirm that the counter shows ten students and both red and green points are visible.
3. Press **Step 1 Iteration** once. Read the active point p , target t , old score n , output a , error $e = t - a$, weights W , and bias b .
4. Continue stepping or press **Train (Animated)**. A dashed ring marks the sample being visited. The line and blue weight arrow move only after a mistake.
5. Stop when the badge says **Converged - 100% correct**. Read the number of epochs and corrections in the narration.
6. Repeat with XOR-like data. The expected result is **No single line fits**; this is a model-capacity lesson, not a server failure.

1.6.4 How to read every output

Table 1.5: Perceptron outputs in plain language.

Output	Interpretation
$W = [w_1, w_2]$	Two steering numbers. Their direction is perpendicular to the line and points toward the predicted Pass side.
b	The sliding control that moves the line away from the origin.
p	The current student's two feature coordinates.
n	Raw score before thresholding; it is not a probability.
a	Current class, 1 for Pass if $n \geq 0$, otherwise 0.
$e = t - a$	0 means correct; 1 or -1 means a correction was required.
Accuracy	Share of the currently placed examples classified correctly. It is training accuracy, not future-student accuracy.
Boundary	Blue line where $n = 0$; green shade is the predicted Pass half-plane.
Epoch / step	Epoch counts full dataset passes; step counts individual sample visits.

1.6.5 Troubleshooting

If training asks for both classes, add at least one green and one red point. If the line never becomes perfect, test whether a single straight line could separate the colors by eye. If no point appears, click inside the plot. If all initial outputs are Pass, remember that

zero weights give $n = 0$ and this implementation assigns the boundary to class 1. If data is edited, the old line is intentionally cleared because it no longer describes the current points.

1.7 Conclusion

The Perceptron chapter demonstrates the smallest complete supervised learning loop: predict, compare, correct, repeat. Its greatest teaching strength is that the parameters, line, error, and convergence limit are visible. Its greatest limitation is equally visible: one hard-limit neuron can draw only one linear boundary.

Lab 2

K-Nearest Neighbors (KNN)

Problem: Adult BMI Category from Five Nearby Examples

2.1 Problem Description

The BMI Neighbor project accepts adult height in centimetres and weight in kilograms, calculates Body Mass Index, and lets five saved examples vote for one of four labels: Underweight, Normal, Overweight, or Obese. The purpose is to make instance-based classification inspectable. Cover and Hart established the classic nearest-neighbor classification framework [3].

The categories in the synthetic CSV are organized around CDC adult BMI ranges: below 18.5, 18.5–24.9, 25.0–29.9, and at least 30.0 [4]. The page is restricted to an adult teaching context because children and teenagers require age- and sex-specific interpretation.

Safety boundary: BMI is a screening measurement, not a diagnosis or a complete description of health. This small synthetic dataset is for algorithm learning; personal medical questions belong with a qualified health professional.

2.2 Algorithm Description

2.2.1 The neighbor-vote story

Imagine placing labeled people on a long BMI number line. A new person stands at one BMI value. The computer measures every gap, invites the closest five people, and counts their labels. The most common label wins. If two labels receive the same number of votes, the tied group with the smaller total gap wins.

BMI is computed as

$$\text{BMI} = \frac{\text{weight in kg}}{(\text{height in metres})^2}. \quad (2.1)$$

For 170 cm and 65 kg, BMI is $65/(1.70)^2 \approx 22.49$. The one-dimensional distance from a saved example is

$$d_i = |\text{BMI}_{\text{user}} - \text{BMI}_i|. \quad (2.2)$$

```

1 distances = [(abs(example.bmi - bmi), example)
2               for example in self.examples]
3 distances.sort(key=lambda item: item[0])
4 neighbors = distances[: self.k]
```

Listing 2.1: Distance, sorting, and selection in `knn_model.py`.

Table 2.1: KNN implementation map.

Function / structure	Input / return	Role and calling flow
<code>BmiExample</code>	Four immutable fields	Dataclass holding height, weight, calculated BMI, and known label.
<code>calculate_bmi(h,w)</code>	cm, kg; BMI float	Validates positive values, converts cm to m, and applies the formula.
<code>load_dataset(path)</code>	CSV path; example list	Parses rows, verifies labels, and calculates BMI rather than trusting a stored BMI column.
<code>SimpleKNN(k)</code>	Odd positive integer; model	Stores neighbor count; current value is 5.
<code>fit(examples)</code>	Example list; no value	Training is memory: it copies the labeled examples.
<code>nearest_neighbors(bmi)</code>	BMI; list of $(d, \text{example})$	Measures, sorts, and returns exactly k records.
<code>predict_bmi(bmi)</code>	BMI; label	Counts votes and uses total distance for a tie.
<code>predict(h,w)</code>	cm, kg; dictionary	Calls BMI, neighbor, and vote functions; returns the label and evidence.
<code>export_for_browser(path)</code>	Output path	Writes k and all remembered rows to <code>model.json</code> .
<code>predictWithKnn</code>	BMI and JSON model; object	Browser twin of distance, sorting, voting, and tie-breaking.

KNN is called a lazy learner because `fit` does not compress the data into weights. The exported model is the memory itself: a JSON object containing `k`, `feature`, and an array of training-example objects.

2.3 Dataset Description

The file `KNN/data/bmi_data.csv` contains 60 deterministic, synthetic rows. It was built by choosing five heights (150, 160, 170, 180, and 190 cm) and twelve target BMI positions around the official adult category boundaries. Weight was calculated using $\text{weight} = \text{BMI} \times (\text{height in metres})^2$ and rounded to one decimal place. Labels follow the CDC ranges [4]. This creates examples on both sides of important boundaries without using patient records.

Table 2.2: KNN CSV columns and verified contents.

Column	Observed range	Meaning
height_cm	150–190	Synthetic adult height in centimetres; used to calculate BMI and draw the map.
weight_kg	37.1–151.6	Synthetic weight in kilograms; used to calculate BMI and draw the map.
category	Four words	Known teaching label. Counts: 10 Underweight, 15 Normal, 15 Overweight, and 20 Obese.
Calculated bmi	approximately 16.5–42.0	Derived at load time, then included in the exported JSON.

The dataset is deliberately small and regular. This helps a learner see the neighbors, but it does not provide demographic diversity or an independent clinical validation sample.

2.4 Training

```

1 cd C:\Users\User\machine_learning\KNN
2 python train_model.py
3 python start_app.py

```

Listing 2.2: Train, export, and serve the KNN project.

The training script loads 60 rows, places every fifth row into a 12-row test pile, lets a temporary model remember the remaining 48 rows, and measures held-out accuracy. The verified result was 100% on those 12 synthetic rows. It then fits a final $k = 5$ model on all 60 rows and writes `dist/model.json`. The browser loads that JSON and repeats the same distance-and-vote rule locally.

Table 2.3: KNN parameters and their effects.

Choice	Current	What changes
k	5	$k = 3$ follows very local examples and can react to noise; $k = 7$ smooths the vote but may blur a narrow category. Use a positive odd value and retrain/export.
Distance feature	BMI only	Height and weight appear on the graph, but neighbors are chosen by BMI gap. A two-feature distance would be a different model and would require scaling.
Tie rule	Total gap	If vote counts tie, the category whose voters have the smaller combined BMI distance wins.
Dataset rows	60	Added rows can change which five examples are nearest. Keep labels consistent, then rerun <code>python train_model.py</code> .
Split rule	Every fifth row	This deterministic check is reproducible but not a substitute for repeated or external validation.

2.5 Website Design and Answer Flow

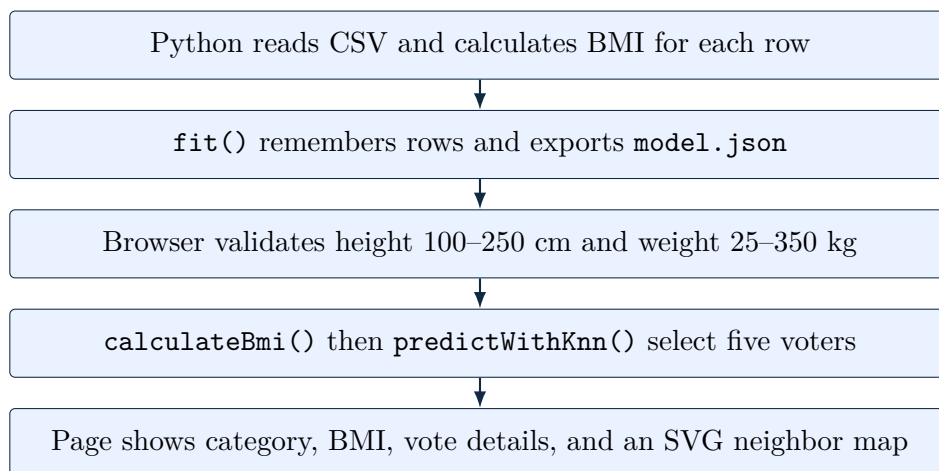


Figure 2.1: KNN training-to-browser flow.

Prediction is entirely client-side after the static files load. The SVG uses height on the x axis and weight on the y axis, but the five ringed points are chosen by BMI distance. Dashed lines are visual links, not the metric used by the classifier.

2.6 Comprehensive User Manual

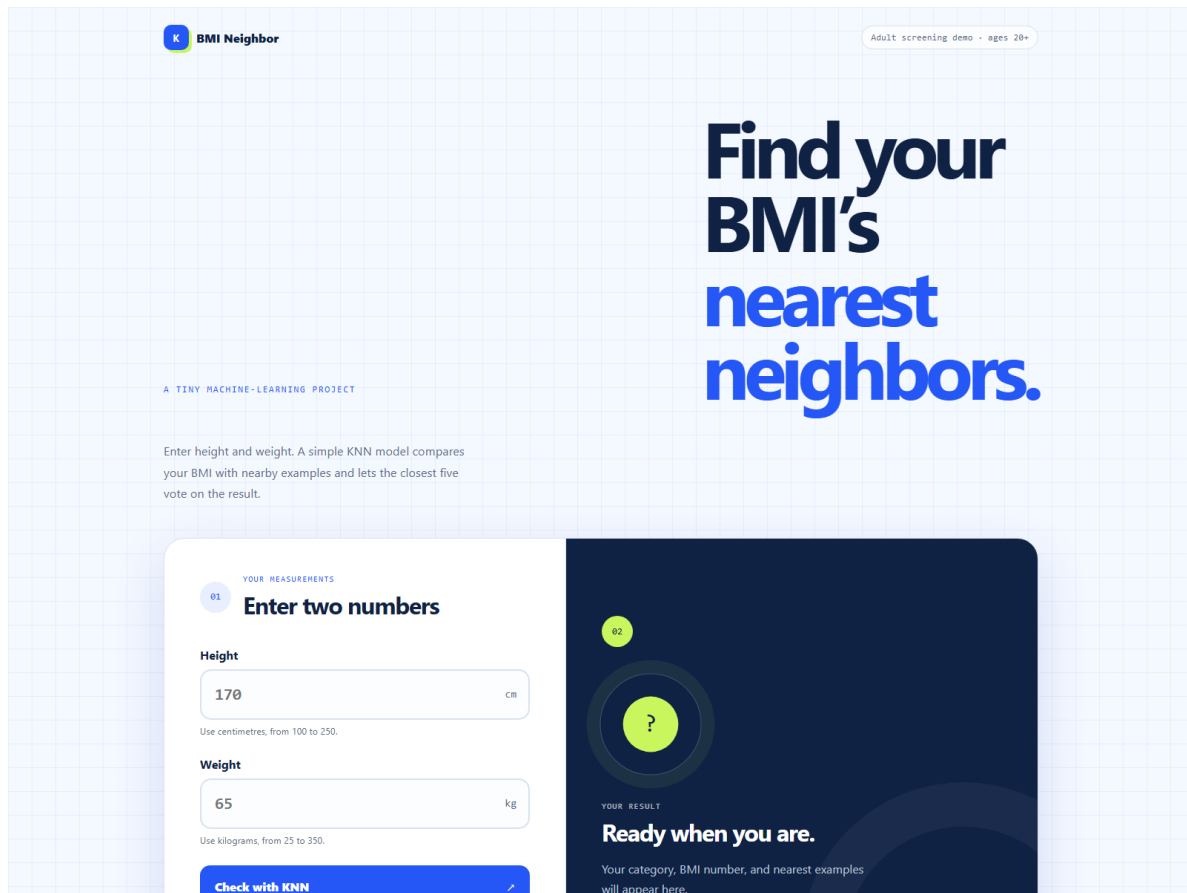


Figure 2.2: BMI Neighbor website with the two measurement inputs and result panel.

2.6.1 Inputs and buttons

1. Enter adult height from 100 to 250 cm. Enter the number measured from the floor to the top of the head, without shoes when possible.
2. Enter weight from 25 to 350 kg. Use kilograms, not pounds.
3. Optionally press the 170 cm/65 kg or 170 cm/100 kg example button. Presets fill the same form and do not bypass validation.
4. Press **Check with KNN**. The button remains disabled until the JSON model is loaded.
5. Use **Reset** to restore the initial example and clear the result.

2.6.2 Outputs

The result card reports the calculated BMI to one decimal place and one winning category. The category colors are reading aids only. The neighbor panel lists five voters with rank,

height, weight, BMI, label, and distance. On the graph, the lime diamond is the entered person; small colored dots are all 60 saved examples; large numbered rings are the five voters.

Table 2.4: KNN result meanings.

Displayed item	Meaning
Underweight / blue	Five nearby synthetic BMI examples voted for the below-18.5 teaching category.
Normal / green	The vote selected the 18.5–24.9 teaching category. The CDC uses the term Healthy Weight; the code’s literal label is Normal.
Overweight / amber	The vote selected the 25.0–29.9 teaching category.
Obese / red	The vote selected BMI at least 30; it is not a diagnosis or statement about the whole person.
BMI number	A height/weight ratio, not a percentage.
Distance	Absolute difference between two BMI values. Smaller means nearer.
Neighbor rank	1 is closest; 5 is the last voter included.

2.6.3 Worked example

For 170 cm and 65 kg, the page calculates $\text{BMI} \approx 22.5$. It compares 22.5 with all 60 saved BMI values, sorts the absolute differences, and lets the first five labels vote. The expected teaching label is Normal. A user should report this as “the five nearest synthetic examples voted for Normal,” not “the person is medically normal.”

2.6.4 Troubleshooting

If the button stays disabled, wait for the model-ready state and confirm the page is served through `start_app.py`; opening `index.html` directly can block `fetch('model.json')`. If a field is red, check its unit and range. If a visually distant graph point becomes a neighbor, remember that KNN uses BMI, so different height/weight pairs can share almost the same BMI. After editing the CSV, always rerun `python train_model.py`.

2.7 Conclusion

This project exposes KNN’s defining idea: training stores examples and prediction performs the real work. The visible neighbor evidence makes the answer auditable, while the synthetic and one-feature design makes the limits clear.

Lab 3

Decision Tree

Problem: Explainable Cardiac-Triage Demonstration

3.1 Problem Description

This laboratory asks a deliberately small question: can a computer turn two numbers on a synthetic triage card into a visible chain of yes/no decisions? The two inputs are systolic blood pressure (BP), measured in mmHg, and maximum heart rate (HR), measured in beats per minute. The output is the teaching label **low** or **high**. The useful part is not a medical verdict; it is the ability to see every rule that the program followed.

Safety boundary: The 48 records and their labels are synthetic. Real chest-pain and cardiac assessment uses symptoms, history, examination, ECG, repeated blood tests such as troponin, and sometimes imaging. A two-input classroom tree must never guide treatment or delay emergency care [5, 6].

3.2 The Algorithm

3.2.1 A tree made from questions

A decision tree resembles the game “Twenty Questions.” At an internal node, the program asks a question such as “Is BP at most 140?” A yes answer follows the left branch; a no answer follows the right branch. A leaf is an ending box that stores the class to return. This project’s complete learned tree is:

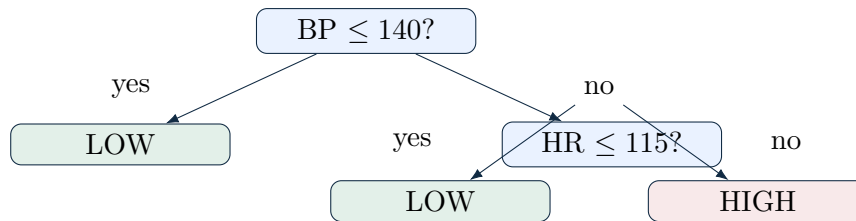


Figure 3.1: The exact tree exported by the current training script.

Plain meaning. For BP 155 and HR 130, the first answer is no and the second is also no, so the route ends at HIGH. A prediction is simply a walk from the top box to one ending box.

3.2.2 How the program chooses a question

The CART method repeatedly looks for a split that makes the child groups purer than their parent [7]. For a node with class proportions p_c , its Gini impurity is

$$G = 1 - \sum_c p_c^2. \quad (3.1)$$

A box containing only one label has $G = 0$. A mixed half-low, half-high box has $G = 1 - (0.5^2 + 0.5^2) = 0.5$. For each feature, the implementation sorts its unique values and tests the midpoint between every neighboring pair. A candidate's score is the size-weighted impurity of its two children:

$$G_{\text{split}} = \frac{n_L}{n} G_L + \frac{n_R}{n} G_R. \quad (3.2)$$

The candidate with the smallest score becomes the question. The routine then calls itself once for the left subset and once for the right subset. Recursion stops when all labels match, depth reaches `max_depth`, too few records remain, or no useful split exists. The leaf uses the majority label.

```

1 for feature in ("bp", "hr"):
2     values = sorted({row.value(feature) for row in rows})
3     thresholds = [(a + b) / 2 for a, b in zip(values, values[1:])]
4     for threshold in thresholds:
5         left = [r for r in rows if r.value(feature) <= threshold]
6         right = [r for r in rows if r.value(feature) > threshold]
7         score = (len(left) * gini_impurity(left)
8                 + len(right) * gini_impurity(right)) / len(rows)
9         if score < best_score:
10             best = (feature, threshold, left, right)

```

Listing 3.1: Core split search, simplified from `decisionTree/decision_tree_model.py`.

The strict less-than comparison in the last line gives deterministic tie handling: the first equally good candidate remains selected. The row object is a `TrainingExample`; a `TreeNode` holds an ID, sample counts, prediction, optional feature and threshold, plus

left/right child references. Export converts references into nested JSON so JavaScript can read the tree.

Table 3.1: Decision-tree functions and data flow.

Function	Input and parameters	Output or call
<code>load_dataset</code>	CSV path	List of typed <code>TrainingExample</code> objects
<code>gini_impurity</code>	A list of rows	One impurity number
<code>_best_split</code>	Rows at the current node	Best feature, threshold, and two row lists
<code>_build</code>	Rows and current depth	Calls itself for children; returns a <code>TreeNode</code>
<code>fit</code>	All training rows	Resets node IDs and stores the root
<code>predict</code>	One BP and HR pair	Walks the stored nodes and returns a label
<code>export</code>	Fitted model and output path	JSON tree, labels, rules, ranges, and accuracy

3.3 Dataset Description

The file `decisionTree/data/cardiac_triage.csv` contains 48 teaching cards. It was created as the Cartesian product of eight BP levels (90, 105, 120, 135, 145, 155, 170, 180) and six HR levels (60, 80, 100, 120, 140, 160). This balanced grid makes the two-dimensional rule easy to inspect; it is not a patient sample. There are 36 **low** and 12 **high** labels.

Table 3.2: Meaning of each cardiac demonstration column.

Column	Stored values	Plain-language meaning
<code>systolic_bp</code>	90–180 mmHg	Pressure when the heart contracts; it is feature one.
<code>max_heart_rate</code>	60–160 bpm	A synthetic maximum-rate value; it is feature two.
<code>triage_label</code>	low/high	The answer supplied to the learner, not a diagnosis.

The labels were arranged so only cards with BP above 140 and HR above 115 are **high**. Training rediscovers that design. Authoritative guidance is cited to show why real assessment is much richer, not to claim that the synthetic labeling rule is clinical guidance [5, 6].

3.4 Training

From the workspace, run:

```
1 cd decisionTree
2 python train_model.py
3 python start_app.py
```

The first command loads the CSV and creates a `SimpleDecisionTree` with maximum depth 2 and minimum split size 2. It then calls `fit`, measures predictions on all 48 rows, and writes `dist/model.json`. The verified result was 100% training accuracy. That only confirms the tree reproduces this designed grid; there is no held-out clinical validation.

- Raising `max_depth` permits more questions and finer regions, but can memorize noise. Lowering it to 1 removes the HR question.
- Increasing `min_samples_split` refuses to divide small groups, producing a shorter, smoother tree.
- Changing CSV labels moves the best thresholds. Retrain after every dataset edit or the browser will continue using old JSON.

3.5 Website

The server only hosts static files; the already-trained JSON is the browser's source of truth. JavaScript validates the node structure, walks the tree, records the visited path, computes colored decision regions, and redraws the SVG without sending the entered numbers to Python.

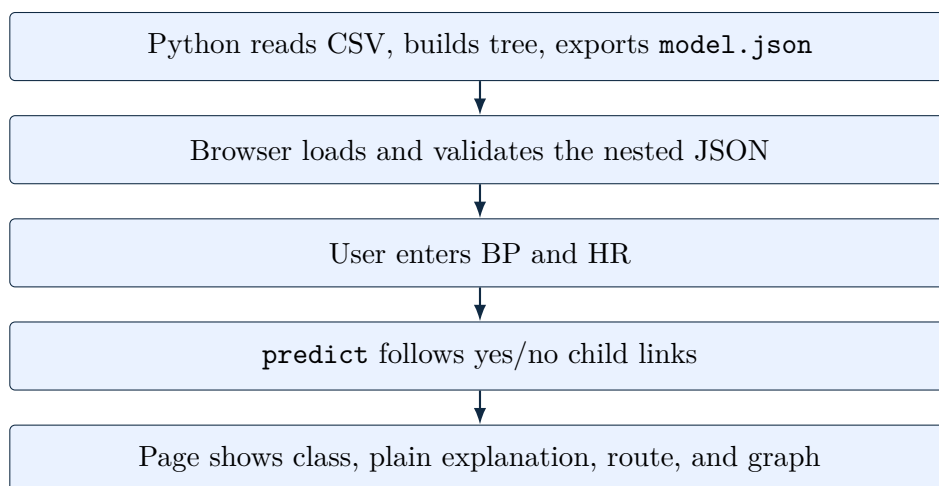


Figure 3.2: Decision-tree training and browser prediction flow.

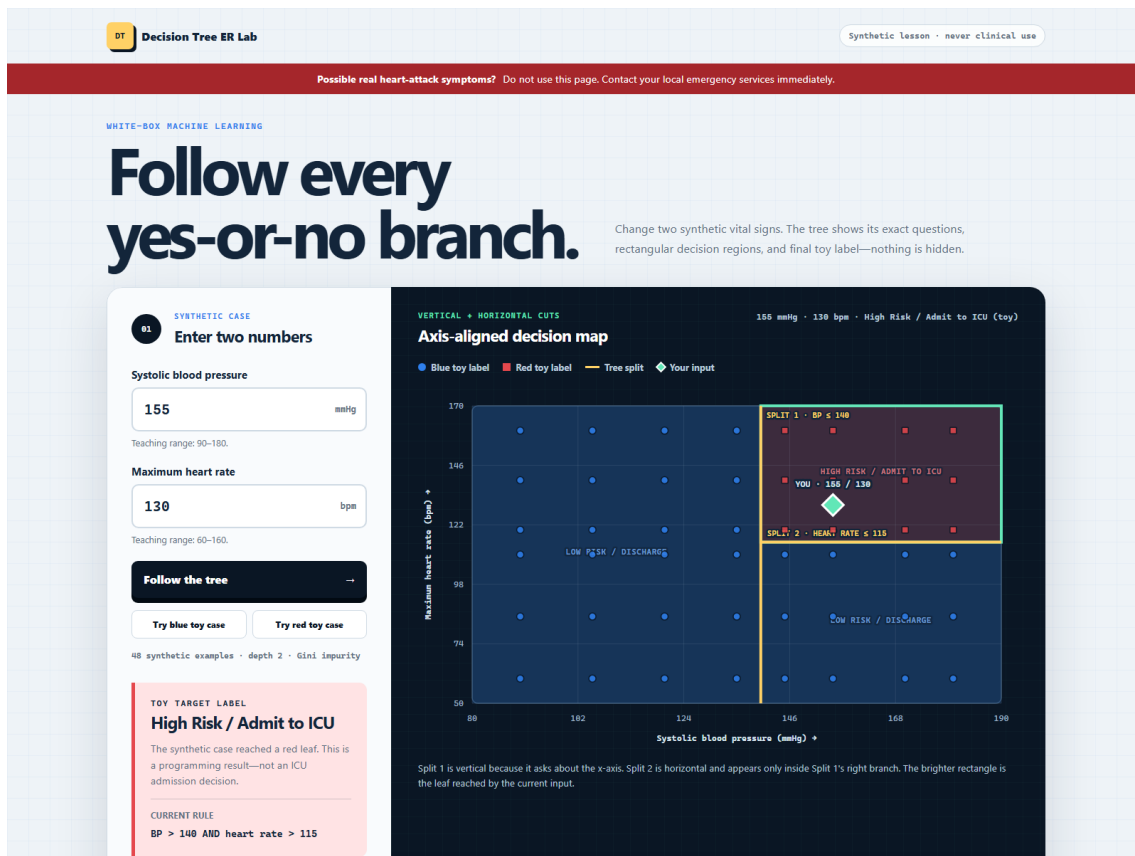


Figure 3.3: Live decision-tree page showing a HIGH route for BP 155 and HR 130.

3.6 Comprehensive User Manual

3.6.1 Start and enter a card

Run `python start_app.py`, open the printed local address, then enter:

1. systolic BP from 90 to 180;
2. maximum HR from 60 to 160;
3. press **Follow the tree**. The preset buttons fill one low-route or one high-route example; reset restores the starting state.

3.6.2 Read every output

- **LOW/HIGH** is the synthetic leaf label, not a risk percentage.
- **Decision path** restates each comparison and the entered value. A yes/no marker tells which branch was taken.
- **Tree diagram** highlights visited boxes, letting a reader audit the same route visually.

- **Decision map** colors every BP–HR region and plots the current card. Its vertical and horizontal borders are learned thresholds.
- **Training accuracy** describes the 48 design cards only.

For the screen in Fig. 3.3, $155 > 140$ and $130 > 115$, hence HIGH. If no result appears, wait for “Model ready,” serve the `dist` folder instead of opening the HTML file directly, and ensure both entries are in range. If the CSV changed, regenerate JSON and refresh the page.

3.7 Conclusion

This lab makes model interpretability concrete: the answer is the leaf and the explanation is the route. Its very simplicity also teaches an important limit: perfect performance on designed examples is not evidence of medical validity.

Lab 4

Support Vector Machine

Problem: Diabetes-Screening Margin Visualizer

4.1 Problem Description

This project draws a straight separating road between two synthetic groups of fasting-glucose and BMI examples. Its trained classes are **lower** and **higher**. The website also paints a narrow **follow-up** band around the road to explain uncertainty near the margin. That band is a display zone derived from score size, not a third class learned from rows.

Safety boundary: The output is not a diabetes diagnosis. Laboratory diagnosis uses validated tests and clinical interpretation; an abnormal result commonly requires confirmation. NIDDK lists fasting plasma glucose at 99 mg/dL or below as normal, 100–125 as prediabetes, and 126 or above as diabetes, but this synthetic two-feature SVM must not replace professional testing [8].

4.2 The Algorithm

4.2.1 A wide road between two groups

A linear support vector machine (SVM) assigns a signed score

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{z} + b, \quad z_j = \frac{x_j - \mu_j}{s_j}, \quad (4.1)$$

where $\mathbf{x}$ contains glucose and BMI, $\boldsymbol{\mu}$ contains training means, and $\mathbf{s}$ contains training standard deviations. Standardizing prevents the numerically larger glucose scale from automatically dominating. The line $f = 0$ is the center boundary; $f = -1$ and $f = +1$ are margin edges. Negative scores choose **lower**; positive scores choose **higher**.

Unlike a line that merely separates, an SVM prefers a wide gap while penalizing examples that fall inside or across it. The implementation uses labels $y \in \{-1, +1\}$ and repeatedly

reduces the regularized hinge objective

$$\lambda \|\mathbf{w}\|^2 + \frac{1}{n} \sum_{i=1}^n \max(0, 1 - y_i f(\mathbf{x}_i)). \quad (4.2)$$

This is the large-margin principle introduced for support-vector networks by Cortes and Vapnik [9].

```

1 grad_w = 2.0 * regularization * weights
2 grad_b = 0.0
3 for z, y in zip(standardized_rows, labels):
4     margin = y * (dot(weights, z) + bias)
5     if margin < 1.0:                # this row still pushes on the road
6         grad_w -= (y / n) * z
7         grad_b -= y / n
8 weights -= learning_rate * grad_w
9 bias    -= learning_rate * grad_b

```

Listing 4.1: One full-batch update, simplified from `svm_model.py`.

The learning rate slowly decays during 6000 epochs so early steps travel quickly and later steps settle. The final model stores ordinary Python lists for means, scales, weights, labels, ranges, and selected display examples. A `TrainingExample` supplies `features()` and converts the word label to -1 or $+1$.

Table 4.1: Linear-SVM functions and passed values.

Function	Receives	Returns or stores
<code>load_dataset</code>	CSV path	Typed list of 40 examples
<code>fit</code>	Examples plus learning rate, regularization, epochs	Means, scales, two weights, and bias
<code>decision_score</code>	Glucose and BMI	Signed distance-like score f
<code>zone_from_score</code>	Score and display threshold	lower, follow-up, or higher zone
<code>predict</code>	Two raw inputs	Learned binary class from the score sign
<code>support_indices</code>	All trained margins	Closest examples selected for the diagram
<code>export</code>	Fitted model	Browser-readable <code>model.json</code>

4.3 Dataset Description

The SVM data directory contains `diabetes_triage.csv`. It has 40 hand-constructed examples: 20 **lower** rows and 20 **higher** rows. Glucose spans 72–204 mg/dL and BMI spans 19–45 kg/m². The lower group uses fasting-glucose values up to 99, while the higher group begins at 126; the omitted middle interval leaves a visible margin. The thresholds

were inspired by NIDDK’s diagnostic-test explanation [8], but the points, BMI pairings, and group labels are synthetic.

Table 4.2: Meaning of the SVM dataset columns.

Column	Values	Meaning
fasting_glucose	72–204 mg/dL	Blood-glucose concentration after fasting; feature one.
bmi	19–45 kg/m ²	Mass relative to squared height; feature two.
screening_label	lower/higher	Supplied teaching class used as -1 or $+1$.

4.4 Training

```

1 cd svm_diabetes_triage
2 python train_model.py
3 python start_app.py

```

The trainer standardizes both columns, starts the two weights and bias at zero, performs full-dataset gradient steps, measures training accuracy, chooses near-margin examples for display, and exports JSON. The verified current result is 100% on all 40 design rows. The exported parameters are approximately $\mathbf{w} = (2.5947, 0.0548)$, $b = 0.6838$, $\boldsymbol{\mu} = (122.5875, 31.725)$, and $\mathbf{s} = (37.8987, 7.7168)$.

- A larger learning rate than 0.08 moves faster but can jump back and forth; a smaller value is steadier but needs more epochs.
- More regularization than 0.02 shrinks the weights and favors a wider, simpler boundary; too much can underfit.
- More than 6000 epochs helps only until updates have settled. Always check loss or accuracy instead of assuming that more is better.
- The website’s follow-up width changes interpretation near $f = 0$ but does not retrain the two-class SVM.

4.5 Website

The Python stage writes the learned numbers once. On page load, JavaScript checks that JSON contains finite two-element arrays. For each user pair it standardizes with the saved means/scales, calculates the dot product and bias, maps the score to a display zone, and moves the marker on the SVG.

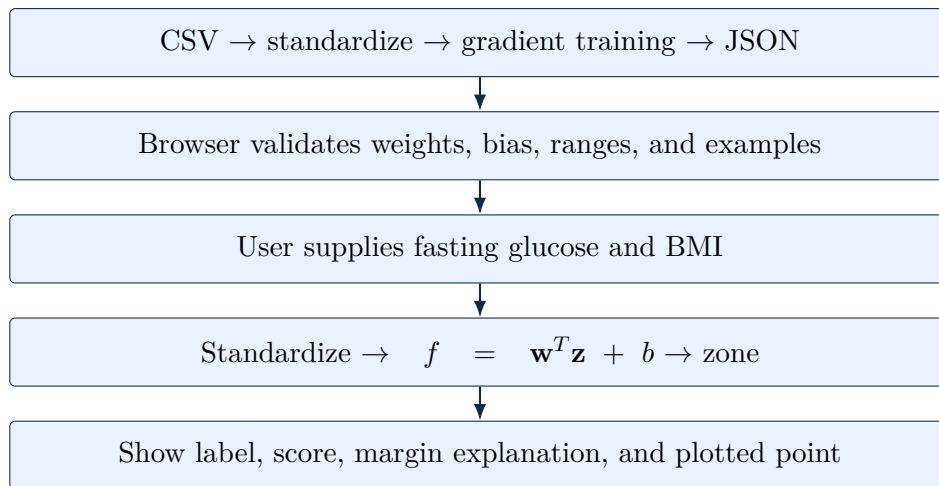


Figure 4.1: SVM training, export, and browser scoring flow.

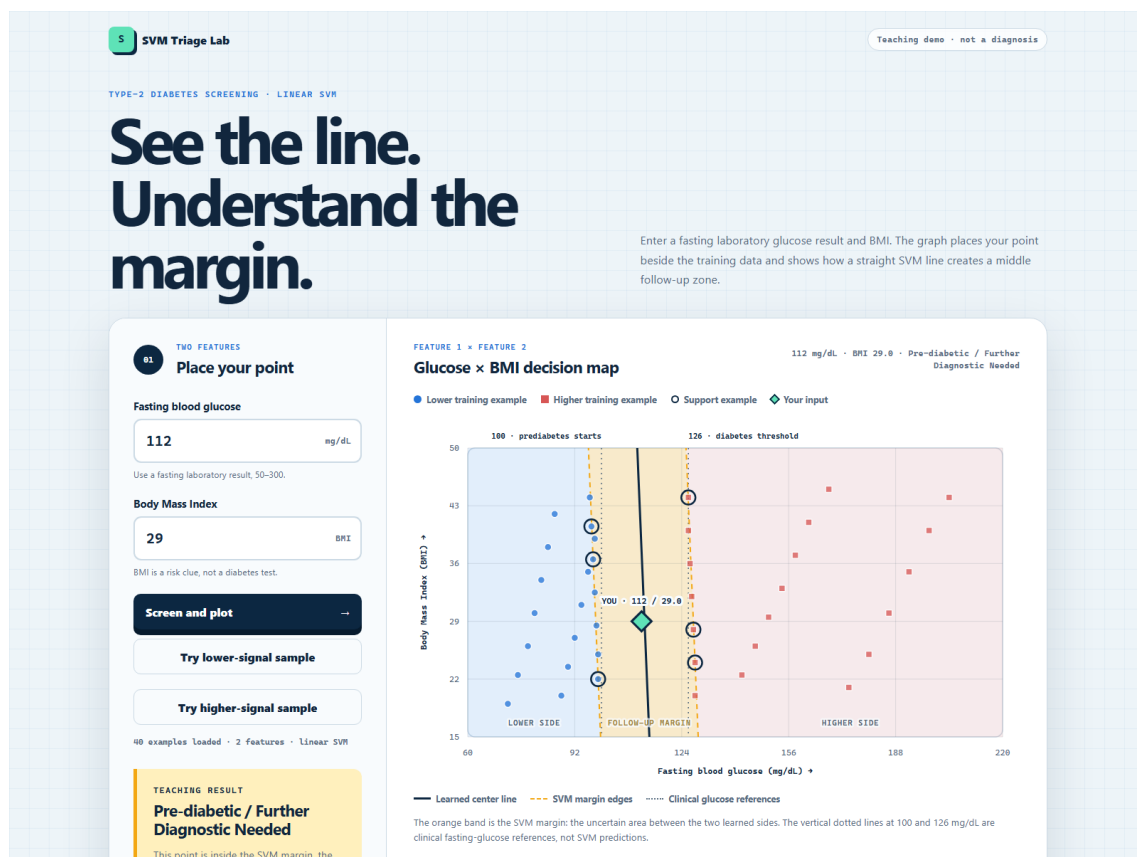


Figure 4.2: Live SVM plot with the higher preset (150 mg/dL, BMI 33).

4.6 Comprehensive User Manual

Start `python start_app.py`. Enter fasting glucose from 50 to 300 mg/dL and BMI from 12 to 60 kg/m², or select a preset, and press **Screen and plot**. Reset restores the demonstration defaults.

- **Lower-side/Higher-side** tells which side of $f = 0$ contains the point; it does not state a probability or diagnosis.
- **Follow-up zone** means $|f|$ is within the configured display margin. It deliberately says the example lies close to the road.
- **Decision score** is signed. Its sign chooses the side and its absolute size measures separation in the model's standardized coordinate system; it is not "percent confidence."
- **Boundary map** shows blue lower and orange higher areas, the central line, margin lines, training points, near-margin examples, and the user's larger marker.
- **Equation/explanation** substitutes the saved parameters so an advanced reader can reproduce the calculation.

In Fig. 4.2, the marker is on the higher side. If loading fails, serve the folder through the start script and regenerate `model.json`. If an input is rejected, check units and allowed range. Treat any health concern as a reason to use proper clinical care, never as a reason to tune this demo.

4.7 Conclusion

The SVM project links geometry, optimization, and explanation: standardized features enter one equation and its sign selects a side. The visible margin also demonstrates why near-boundary answers deserve caution.

Lab 5

K-Means Clustering

Problem: Customer-Segment Explorer

5.1 Problem Description

A shop may have customers but no ready-made group labels. This project asks the computer to discover two patterns from each fictional customer's age and spending score. It then lets a user place a new customer on the same map and see the nearer group. The two friendly names, *Budget Explorers* and *Premium Loyalists*, are attached after clustering according to which center has the lower or higher spending score; they are interpretations, not labels shown during learning.

5.2 The Algorithm

5.2.1 Move two team flags until nobody changes team

K-Means begins with K center points. It alternates two simple steps:

1. **Assignment:** send every point to its nearest center.
2. **Update:** move each center to the arithmetic mean of its team's coordinates.

The cycle ends when no assignment changes or a safety limit is reached. This is the iterative clustering idea described by MacQueen [10].

Age and spending use different ranges, so the implementation first compares normalized differences. For point $x = (a, s)$ and center $\mu = (a_c, s_c)$,

$$d(x, \mu) = \sqrt{\left(\frac{a - a_c}{52}\right)^2 + \left(\frac{s - s_c}{99}\right)^2}. \quad (5.1)$$

Here 52 is the page's age span (18–70) and 99 is its score span (1–100). Without normalization, a unit of age and a unit of score would be treated as equally large even though their possible ranges differ.

```

1 for (let iteration = 0; iteration < 12; iteration += 1) {
2   const next = customers.map(point =>
3     normalizedDistance(point, centers[0]) <=
4     normalizedDistance(point, centers[1]) ? 0 : 1);
5
6   centers = centers.map((oldCenter, cluster) => {
7     const members = customers.filter((_, i) => next[i] === cluster);
8     return members.length === 0 ? oldCenter : {
9       age: members.reduce((sum, p) => sum + p.age, 0) / members.length,
10      score: members.reduce((sum, p) => sum + p.score, 0) / members.length
11    };
12  });
13  if (next.every((cluster, i) => cluster === assignments[i])) break;
14  assignments = next;
15 }

```

Listing 5.1: The K-Means loop, simplified from `kmeans/dist/app.js`.

An array of 27 objects stores the customers. Another integer array stores team numbers 0 or 1, while a two-object array stores centers. These small structures make assignment by index explicit. The current initial centers are (25, 22) and (56, 76). After two rounds the final groups contain 12 and 15 points, with centers (31.67, 32.08) and (49.27, 65.93) respectively.

Table 5.1: K-Means browser functions.

Function	Receives	Produces or calls
<code>normalizedDistance</code>	A point and a center	One Euclidean distance after scaling
<code>runKMeans</code>	Customer array, starting centers, maximum rounds	Final centers, assignments, and replay snapshots
<code>getTeamRoles</code>	Final centers	Which team gets the budget/premium display name
<code>classifyCustomer</code>	User age and score	Nearest center, segment, distance, and explanation
<code>setInputs</code>	A preset point	Updates both sliders/numbers and validation
<code>startReplay</code>	Saved snapshots and speed	Animates assignment and center movement
<code>draw helpers</code>	Model and selected point	Customer map, boundaries, centers, and legend

5.3 Dataset Description

There is no CSV and no imported customer information. The 27 fictional records are declared directly in `kmeans/dist/app.js`. They were arranged to make two partly separated teaching groups across the accepted ranges. Because K-Means is unsupervised, no answer column is provided.

Table 5.2: Fields in each fictional customer object.

Field	Range	Plain-language meaning
age	18–70 years	Horizontal position of a fictional adult customer.
score	1–100	Artificial shopping-engagement/spending score; a larger number means stronger activity in this demonstration.
No label	—	The missing answer is the point: K-Means must form teams itself.

To recreate or expand the data, add objects such as `{age: 35, score: 44}` to the `customers` array, keep values in range, reload, and inspect whether the final grouping is stable. Real customer segmentation would require lawful, representative, privacy-conscious data and a business definition of a useful segment.

5.4 Training

Training occurs in JavaScript as soon as the page loads; no Python model file exists. From the project folder:

```

1 cd kmeans
2 npm start
3 # Open the printed localhost address; loading the page runs K-Means.
```

The current run stabilizes after two rounds. To retrain, change the customer array, starting centers, or algorithm settings and refresh.

- K is fixed at 2. Supporting $K = 3$ requires a third center, a general nearest-center loop, three colors/names, and UI changes.
- Different starting centers can lead to a different local solution. Testing several starts and keeping the smallest within-cluster squared error is a common improvement.
- The maximum of 12 rounds is only a guard. Raising it matters if a modified dataset has not stabilized; it does not force extra movement after convergence.
- Removing normalization gives the feature with the larger numerical variation more influence. Changing the age/score spans changes the meaning of distance.

5.5 Website

The entire pipeline is local to the page. The saved snapshots make an animation possible without rerunning a different model during replay.

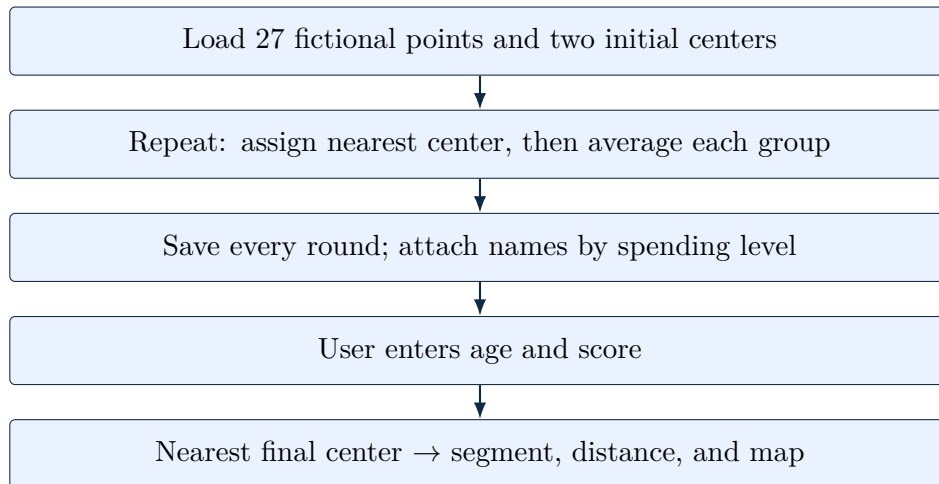


Figure 5.1: K-Means page-load training and user classification flow.

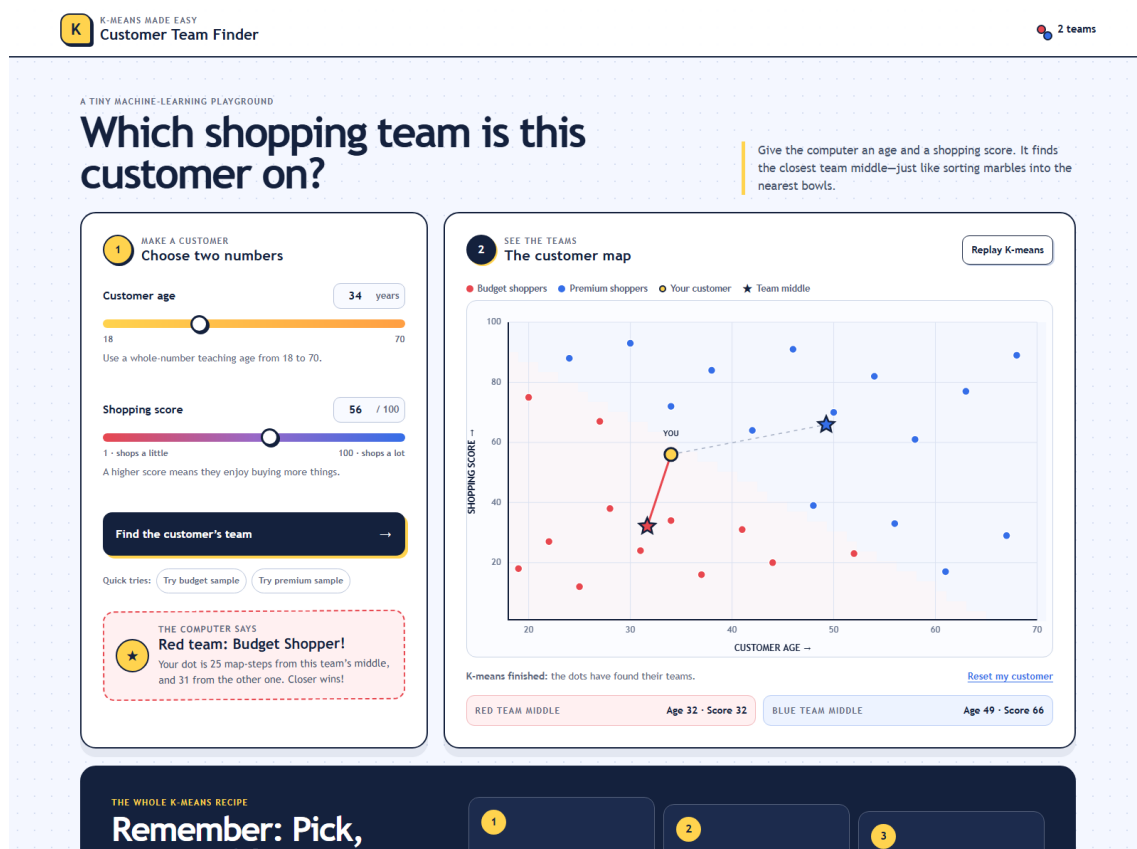


Figure 5.2: Live customer explorer after classifying age 48 and score 86.

5.6 Comprehensive User Manual

1. Start with `npm start` and open the page.
 2. Enter an age from 18 to 70 and score from 1 to 100, or use a preset. Inputs are linked to sliders, so moving either representation updates the other.
 3. Press **Classify customer**. Use **Replay K-Means** to watch center movement; replay changes the visualization, not the final model.
 4. Press reset to restore the original example and camera state.
- **Segment name** is a friendly description assigned after clustering. It is not a moral judgment and was not present in training.
 - **Cluster number/color** is the algorithm's team identity. Swapping numerical identities changes no grouping meaning.
 - **Centroid** is the average age and score of all current members. It may not be an actual customer.
 - **Distance** is the normalized separation between the new point and its selected centroid; smaller means more similar under these two fields. It is not probability.
 - **Map** plots data dots, centroid stars, the decision boundary, and the entered point. Hover details identify values and team.
 - **Iteration cards** explain which round changed assignments.

Figure 5.2 places age 48, score 86 with the higher-spending center. If nothing draws, use a current browser with JavaScript enabled and serve the page rather than relying on restrictive local-file access. If a number is red, bring it inside the printed range.

5.7 Conclusion

This explorer clearly separates discovery from naming: K-Means finds compact groups using distance, then people decide whether those groups are meaningful. The replay turns an abstract iterative calculation into visible movement.

Lab 6

Hidden Markov Model

Problem: Hidden Coin Detective

6.1 Problem Description

Imagine that a person secretly chooses between a fair coin and a heads-biased coin. The coin itself is hidden, but the observer records each Head (H) or Tail (T). The task is to infer the most likely hidden coin used at every toss and to show how uncertainty changes through time. This is a sequence problem: the order of observations matters.

6.2 The Algorithm

6.2.1 Model pieces

A hidden Markov model (HMM) contains hidden states, visible symbols, and three probability structures [11]:

- π : starting probabilities for Fair and Biased;
- A : transition probabilities, such as Fair staying Fair;
- B : emission probabilities, such as Biased producing H.

With default controls the app constructs

$$\pi = (0.5, 0.5), \quad A = \begin{bmatrix} 0.8 & 0.2 \\ 0.2 & 0.8 \end{bmatrix}, \quad B = \begin{bmatrix} 0.5 & 0.5 \\ 0.85 & 0.15 \end{bmatrix}, \quad (6.1)$$

where emission columns are H and T. A Markov assumption says the next hidden coin depends on the current coin, not the full earlier history. An emission depends on the current hidden coin.

6.2.2 Viterbi: the single best hidden route

Multiplying many probabilities can underflow toward zero, so the implementation adds logarithms. Let $\delta_t(j)$ be the best log-score for all routes ending in state j at time t . Then

$$\delta_1(j) = \log \pi_j + \log b_j(o_1), \quad (6.2)$$

$$\delta_t(j) = \max_i [\delta_{t-1}(i) + \log a_{ij}] + \log b_j(o_t). \quad (6.3)$$

For every maximum, a backpointer $\psi_t(j)$ stores which earlier state won. After the final toss, following those pointers backward reconstructs the best path. With T observations and N states, work grows as $O(TN^2)$.

```

1 for time in range(1, len(observations)):
2     for current in range(state_count):
3         choices = [score[time - 1][previous]
4                     + log(transition[previous][current])
5                     for previous in range(state_count)]
6         winner = max(range(state_count), key=lambda p: choices[p])
7         score[time][current] = (choices[winner]
8                                 + log(emission[current][symbol]))
9         backpointer[time][current] = winner
10 # Start at the best final state and follow backpointers toward time zero.
```

Listing 6.1: Viterbi recursion, simplified from HMM/dist/hmm.py.

6.2.3 Forward-backward: belief at each position

The best complete route is not the same as the probability of each state at one position. Forward values summarize all ways to reach a state; backward values summarize all possible futures. Their product is normalized to obtain $P(q_t = j \mid o_1, \dots, o_T)$. The website therefore reports both the Viterbi route and a posterior Fair/Biased percentage at every toss.

Table 6.1: HMM functions, inputs, and outputs.

Function	Receives	Produces
<code>cleanObservations</code>	Text tokens	Validated H/T array of length 1–60
<code>checkProbability</code>	Name and numeric value	Safe probability or a clear error
<code>makeModel</code>	Heads chances and stay chances	π , A , and B arrays whose rows sum to one
<code>viterbi</code>	Observation array and model	Best state-index path and path probability
<code>forwardBackward</code>	Same sequence and model	Posterior pair for every position
<code>analyse</code>	Raw observations and four settings	Named path, summaries, posteriors, and model
<code>renderResult</code>	Analysis object	Route cards, timeline, chart, and explanation

JavaScript performs the live calculation; `HMM/dist/hmm.py` is a matching Python teaching implementation used by the tests. Both pass arrays and plain objects explicitly, which makes every probability inspectable.

6.3 Dataset Description

This lab has no stored training table. Its “data” is one observation sequence entered by the user plus four probability settings. Spaces, commas, or a compact string such as `HHHHTHTT` are accepted and converted to one H/T array.

Table 6.2: HMM input data and model controls.

Item	Allowed/default	Plain-language meaning
Observation	H or T, 1–60 tosses	What the observer can actually see, in order.
Fair heads chance	0.50 default	Chance that Fair emits H; its T chance is one minus this.
Biased heads chance	0.85 default	Chance that Biased emits H.
Fair stay chance	0.80 default	Chance the hidden source remains Fair for the next toss.
Biased stay chance	0.80 default	Chance it remains Biased for the next toss.
Initial state	0.50 each	Before seeing a toss, both coins are equally likely.

The supplied presets (heads-heavy, tails-heavy, and alternating) are synthetic sequences

chosen to reveal behavior. They were not collected from experiments.

6.4 Training and Parameter Experimentation

This website does *not* train probabilities and does not implement the Baum-Welch algorithm. It performs decoding and inference using probabilities the user supplies. This distinction prevents calling every model calculation “training.” To run it:

```
1 cd HMM
2 python -m http.server 4173 --directory dist
3 # Then open http://localhost:4173
4
5 # Optional Python calculation from inside HMM/dist:
6 python -c "from hmm import analyse_json; print(analyse_json('H H T'))"
```

Changing a heads probability changes what each state tends to emit. For example, raising Biased heads chance makes a long H run stronger evidence for Biased. Raising a stay probability penalizes frequent switching and produces longer same-state segments. Lowering stay probabilities makes alternating states easier. A probability of exactly zero would make a path impossible; the UI keeps sliders between 5% and 95% to avoid extreme numerical and teaching cases.

6.5 Website

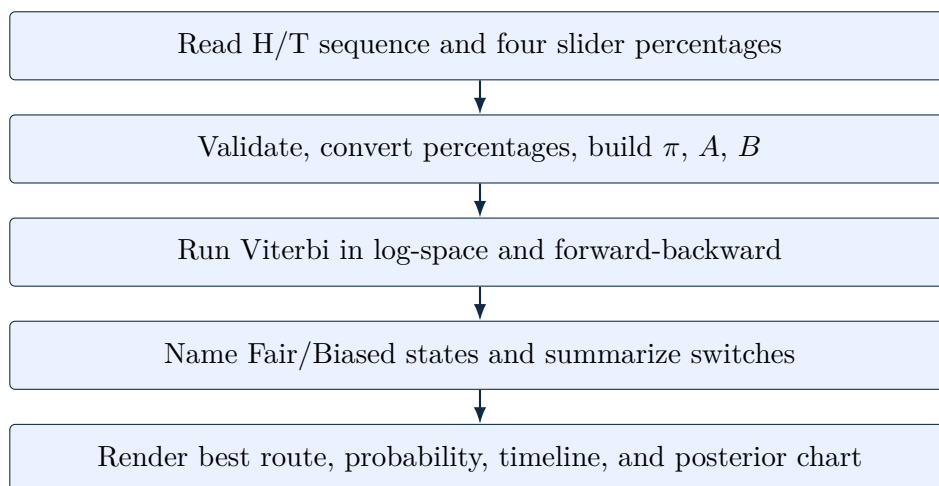


Figure 6.1: Hidden Coin Detective inference flow.

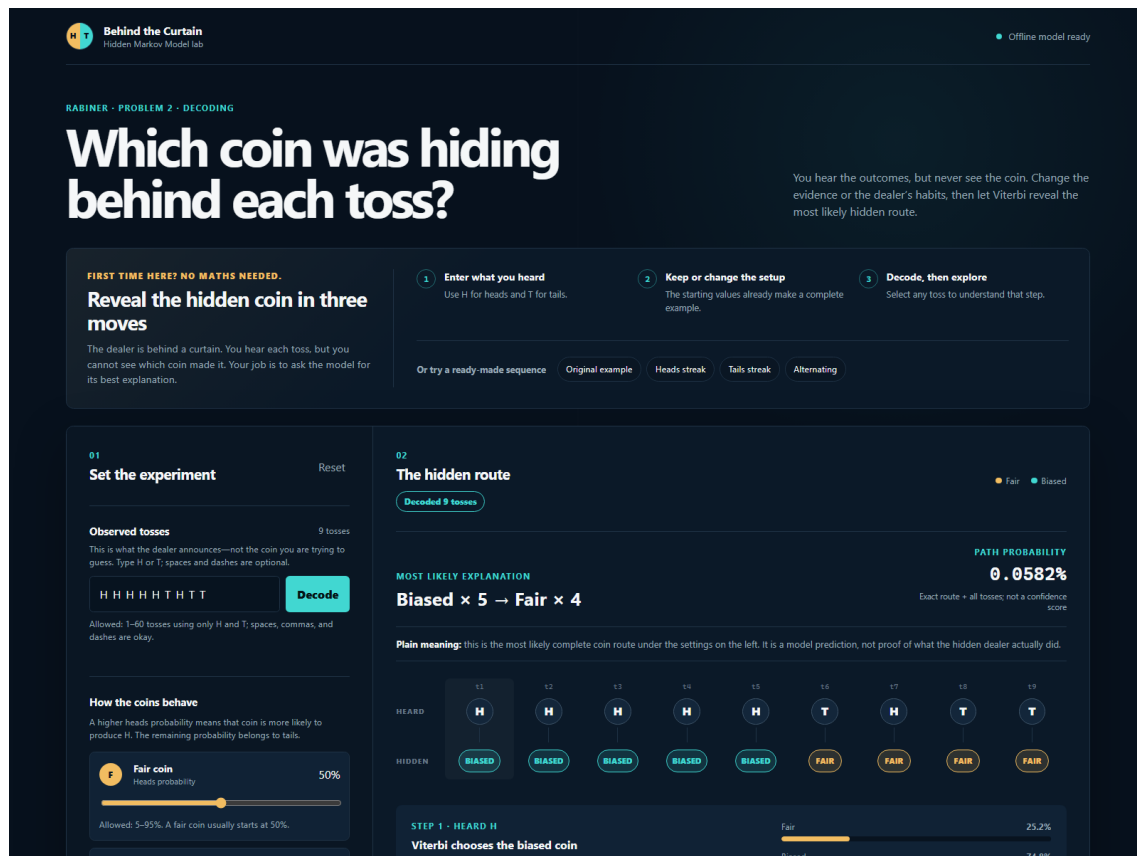


Figure 6.2: Live HMM page decoding the heads-heavy preset.

6.6 Comprehensive User Manual

1. Serve `HMM/dist` and open the page.
 2. Type H and T symbols. Separators may be spaces or commas, or none at all. Do not include other letters. Select a preset to learn by example.
 3. Move the four sliders only if exploring a different coin story. The number beside each slider is the exact percentage used.
 4. Press **Decode hidden coins**. Reset restores the original sequence and default model.
- **Most likely route** is the single complete state sequence with the greatest model probability, for example F–F–B–B. It is an inference, not observed truth.
 - **Path probability** is the joint probability of that route and all observed symbols. It can be small simply because long exact sequences are rare; it is not classifier confidence.
 - **Switch count** says how often adjacent inferred states differ.
 - **Timeline** aligns each visible H/T with its Viterbi state.
 - **Posterior chart** gives Fair and Biased percentages at each position after considering the entire observation sequence. These two bars sum to 100% and can disagree with how decisive the single route looks.

- **Model summary** restates the emissions/transitions actually used, making slider experiments reproducible.

In Fig. 6.2, a heads-heavy run favors the biased coin, while an isolated tail can temporarily shift belief. If decoding is disabled, add at least one valid symbol. If percentages behave unexpectedly, remember that “stay” describes the next hidden state, while “heads” describes the visible outcome from the current state.

6.7 Conclusion

This project shows why sequence models need both memory and uncertainty. Viterbi answers “Which complete hidden story is best?” while forward-backward answers “How plausible is each hidden state here?” Keeping both avoids a common misinterpretation.

Lab 7

CNN and YOLO Object Detection

Problem: Sightline Object Detector

7.1 Problem Description

People can glance at a photograph and say both *what* they see and *where* it is. A computer receives only a rectangular grid of red, green, and blue numbers. This application accepts one photograph or camera frame, finds recognizable objects, draws a box around each one, names it, and reports a confidence score. Unlike ordinary image classification, object detection may return zero, one, or many located answers.

7.2 The Algorithm

7.2.1 From pixels to visual clues

A convolutional neural network (CNN) slides small learned filters over an image. For output position (i, j) and filter k , a simplified operation is

$$y_{i,j,k} = \sigma \left(b_k + \sum_{u,v,c} W_{u,v,c,k} x_{i+u,j+v,c} \right). \quad (7.1)$$

The input x is the pixel array, W and b are learned filter values, and σ is a nonlinear activation. Early layers react to edges and colors; deeper combinations can represent textures, parts, and object shapes. Training has already adjusted millions of such numbers in the supplied weight file.

7.2.2 YOLO detection in one model pass

YOLO means “You Only Look Once.” Its central idea is to predict locations and classes from the whole image in a unified pass rather than running a classifier separately over

many proposed crops [12]. The current project uses the pretrained Ultralytics YOLOv8 nano checkpoint `yolov8n.pt`, a compact member of the YOLOv8 detection family [13].

For each candidate, the network produces four box coordinates, an object class, and a confidence. Low-confidence candidates are discarded. Overlapping guesses for the same object are reduced by non-maximum suppression inside the library; the retained guesses become `result.bboxes`. The application converts each tensor-like box into a plain dictionary:

```

1 results = model.predict(
2     source=image,           # Pillow RGB image
3     conf=confidence,        # accepted range: 0.10 to 0.90
4     imgsz=int(os.getenv("YOLO_IMAGE_SIZE", "640")),
5     verbose=False,
6 )
7 for box in results[0].boxes:
8     class_id = int(box.cls[0].item())
9     score = float(box.conf[0].item())
10    x1, y1, x2, y2 = box.xyxy[0].tolist()
11    detections.append({
12        "label": results[0].names[class_id],
13        "confidence": round(score, 4),
14        "box": [round(x1), round(y1), round(x2), round(y2)]
15    })

```

Listing 7.1: Inference and result conversion from `cnn_object_detector/detector.py`.

Plain meaning. A returned record such as `dog, 0.91, [20,40,500,690]` says: the model guessed “dog” with 91% model confidence, and the rectangle runs from pixel (20,40) to pixel (500,690). Confidence is not a promise that the label is correct.

The `YoloDetector` loads the heavy model only on first use and keeps it for later requests. A re-entrant lock prevents two requests from using the shared model simultaneously. `DetectionResult` is an immutable data class holding annotated JPEG bytes and a list of dictionaries. Detections are sorted strongest-first, assigned visible colors, and passed to `_annotate`, which draws matching boxes and labels on a copy.

Table 7.1: CNN web application functions and information flow.

Function	Receives	Returns or calls
<code>_read_image</code>	Uploaded byte stream	Validated, oriented, bounded RGB Pillow image
<code>_parse_confidence</code>	Form text	Float from 0.10 through 0.90
<code>_get_model</code>	Model filename/environment	Lazily loaded shared YOLO object
<code>detect</code> (detector)	RGB image and confidence	Calls <code>model.predict</code> ; returns <code>DetectionResult</code>
<code>_annotate</code>	Image copy and detection list	JPEG-ready image with colored boxes
<code>/api/detect</code>	Multipart image and confidence	JSON containing data-URL image, boxes, count, time, size, and model
<code>run</code> (browser)	Selected file/camera blob and slider	Calls API, then renders image, cards, and statistics

7.3 Dataset Description

7.3.1 What data the current application uses

The website does not contain a local training CSV and does not add uploaded photos to a dataset. Its learned values came with a model pretrained on the COCO object-detection benchmark. COCO contains complex everyday scenes with large-scale instance annotations [14]; the Ultralytics detection checkpoint exposes the standard 80-category detection label set [13]. Examples include person, bicycle, car, dog, chair, and bottle. An object outside those learned categories can be missed or confused.

The only current per-request “record” is:

Table 7.2: Input image information and its meaning.

Item	Constraint	Purpose
Image bytes	JPG, PNG, WEBP, or BMP; at most 12 MB	The scene to analyze.
Source dimensions	Under 40 megapixels	Blocks decompression-sized images before full loading.
Working dimensions	Longest side at most 1920 before inference	Bounds memory while preserving aspect ratio.
RGB pixels	Three 0–255 color channels	Predictable image representation passed to YOLO.
Confidence	0.10–0.90; default 0.35	Smallest model score allowed into the result.

Phone orientation is corrected from EXIF metadata, then images are converted to RGB using Pillow. Flask’s documented upload pattern motivates receiving the file through multipart form data and applying explicit size/type checks [15]; Pillow performs decoding and orientation [16].

7.4 Training and Model Use

7.4.1 Run the supplied pretrained detector

The normal laboratory workflow is inference, not training:

```

1 cd cnn_object_detector
2 python -m venv .venv
3 .venv\Scripts\Activate.ps1
4 pip install -r requirements.txt
5 python app.py
6 # Open http://127.0.0.1:5000

```

On the first detection, `_get_model` imports Ultralytics and opens the bundled `yolov8n.pt`. The API validates the upload, calls the detector, times it, Base64-encodes the annotated JPEG, and returns one JSON response. The photo is held in memory by this implementation rather than deliberately saved as a server file.

7.4.2 Optional custom training

If a future student creates an annotated YOLO dataset, training is a separate official Ultralytics workflow, for example [17]:

```

1 yolo detect train data=path\to\data.yaml model=yolov8n.pt epochs=50 imgsz=640

```

The YAML identifies training/validation image folders and class names; label text files supply normalized boxes and class IDs. This command is illustrative and does not describe a dataset currently present in the workspace.

- More `epochs` provides more passes over the custom training set, but excessive passes can overfit; validation metrics must decide when to stop.
- Larger `imgsz` can preserve small-object detail but costs more memory and time. The site's inference default is 640.
- A larger batch can speed stable hardware training but needs more memory. Learning-rate changes affect step size and must be tuned carefully.
- On the website, raising confidence (for example 0.35 to 0.60) usually shows fewer, stronger guesses. Lowering it shows more possibilities and more false positives. This filters inference; it does not retrain weights.

7.5 Website

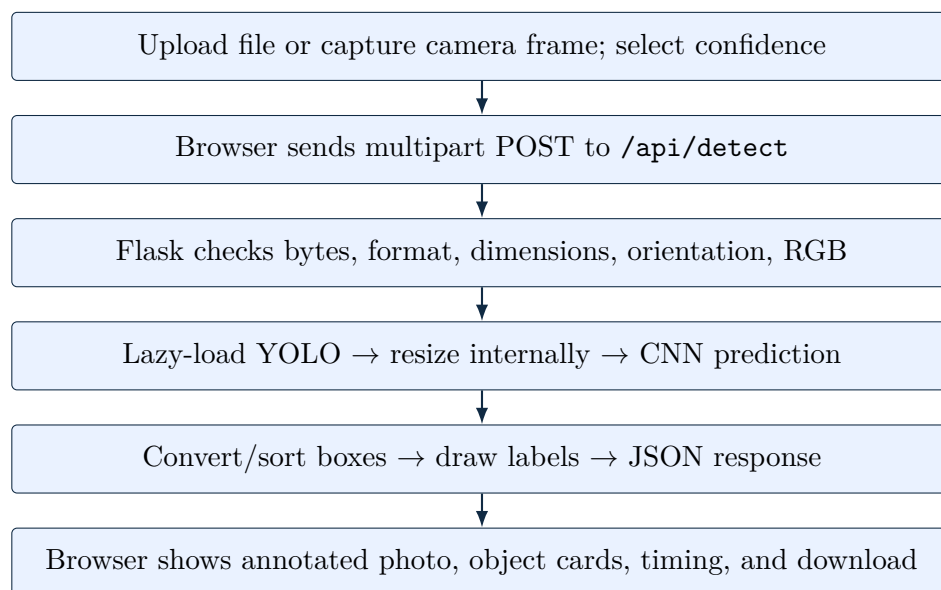


Figure 7.1: End-to-end Sightline request flow.

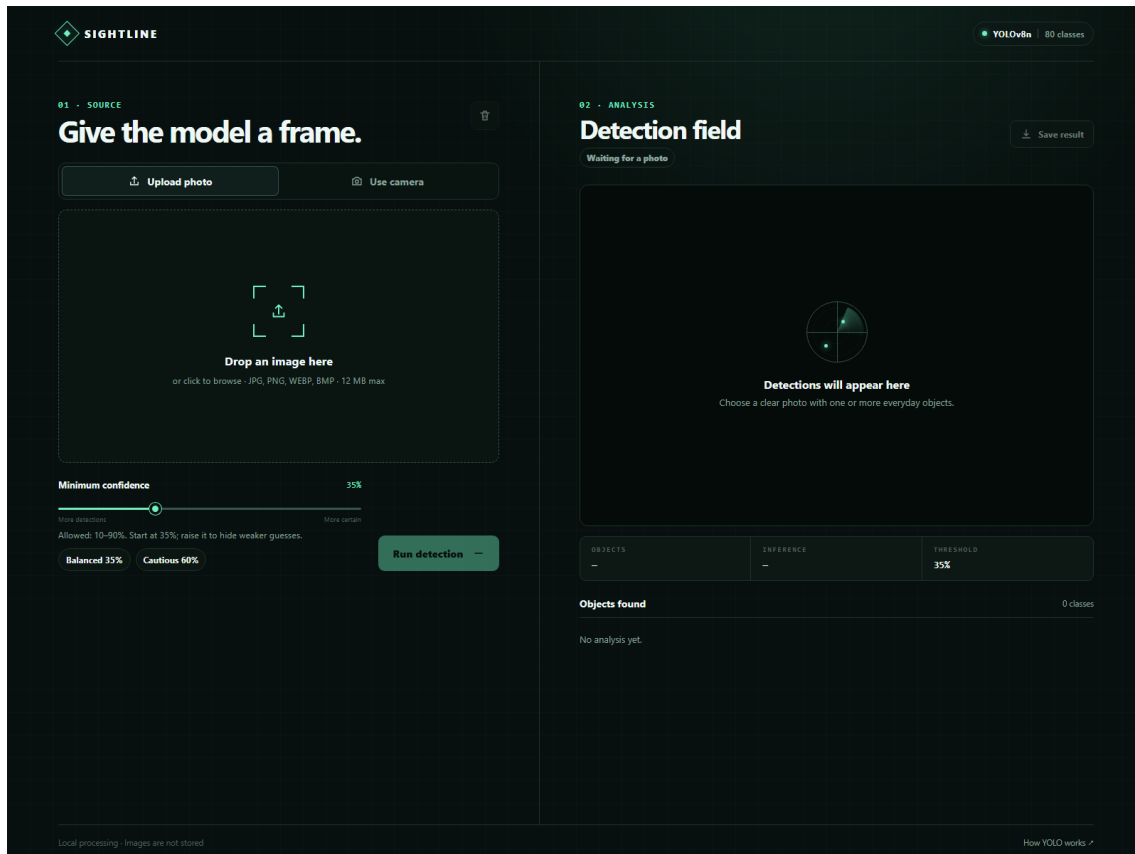


Figure 7.2: Live Sightline upload interface before a photograph is selected.

7.6 Comprehensive User Manual

7.6.1 Choose an input

After starting `python app.py`, open the page and choose one of two tabs:

- **Upload photo:** click or drag a JPG, PNG, WEBP, or BMP no larger than 12 MB. A preview and filename confirm the selection.
- **Use camera:** allow browser camera permission, aim at the scene, and capture a frame. Camera access normally requires localhost or HTTPS.

Do not upload private images to a server you do not trust. This local code does not intentionally save them, but deployment operators remain responsible for transport security, logs, and retention policy.

7.6.2 Run and understand the answer

Set the confidence slider from 10% to 90% (35% is a useful starting point), then press **Detect objects**.

- **Annotated photo:** each colored rectangle is a predicted object location. Its tag contains class name and rounded confidence.

- **Detected objects:** a strongest-first list. Matching colors link list cards to boxes.
- **Count:** number of boxes surviving the chosen confidence and the model’s suppression process, not necessarily the true object count.
- **Time:** server inference elapsed milliseconds. It excludes some browser/network work and varies by CPU/GPU and first model load.
- **Image size/model/threshold:** records the exact request context.
- **Save result:** downloads the annotated JPEG; reset clears the current input and output.

If the output says no objects, try a clear, well-lit photo, reduce confidence a little, or remember that the object may not be one of the 80 known categories. If the model cannot start, activate the environment, install requirements, and confirm `yolov8n.pt` is readable. A 400 error indicates an invalid input; 413 means too large; 503 means the detector could not load/analyze. The first successful run is normally slower because the model changes from “cold” to “loaded.”

7.7 Conclusion

Sightline joins a pretrained visual learner to a defensive, understandable web pipeline. The lab distinguishes pixels, learned CNN features, YOLO detections, application-side validation, and user-facing evidence while making the limits of confidence and label coverage explicit.

Lab 8

Overall Conclusion

The seven laboratories cover complementary ways of learning and reasoning. The Perceptron and SVM draw linear boundaries; KNN consults nearby stored examples; the Decision Tree forms auditable questions; K-Means discovers unlabeled groups; the HMM reasons over hidden sequences; and CNN/YOLO turns pixels into multiple located object hypotheses. Their websites expose not only an answer but also the evidence most natural to each model: weights and a line, neighbors, a branch path, a margin, moving centroids, a state timeline, or bounding boxes.

Across all projects, a model's output inherits the limits of its data and objective. Synthetic labels can teach mechanics but cannot establish clinical truth. Training-set accuracy can verify implementation consistency but cannot replace independent evaluation. A score is not automatically probability, and a cluster is not automatically meaningful. Keeping these distinctions beside the interactive controls is as important as implementing the formula.

Lab A

Verification and Reproduction

A.1 Commands verified in this workspace

The following checks were executed against the submitted sources before this report was produced.

Project	Verification command	Observed result
Perceptron	<code>python -m unittest discover -s tests -v</code>	12 tests passed
KNN	<code>python -m unittest discover -s tests -p test_knn.py -v;</code> <code>node tests/test_browser_model.js</code>	4 Python tests; browser tests passed
Decision Tree	<code>python -m unittest discover -s tests -p test_decision_tree.py -v;</code> <code>node tests/test_browser_tree.js</code>	5 Python tests; browser tests passed
SVM	<code>python -m unittest discover -s tests -p test_svm.py -v;</code> <code>node tests/test_browser_model.js</code>	4 Python tests; browser tests passed
HMM	<code>python -m unittest discover -s tests -p test_hmm.py -v;</code> <code>node tests/test_browser_hmm.js</code>	5 Python tests; browser tests passed
K-Means	<code>node -check dist/app.js</code>	Syntax check passed
CNN/YOLO layer	<code>web python -m pytest tests/test_app.py -q</code>	4 tests passed

The KNN retraining command reported 60 rows, a 48/12 deterministic split, and 100% test accuracy. Decision Tree reported 48 rows and 100% training accuracy. SVM reported 40 rows and 100% training accuracy. These values reproduce the designed laboratory datasets and are not claims of population performance.

A.2 One-page run checklist

1. Install each project's declared dependencies before starting its server.
2. For KNN, Decision Tree, or SVM, rerun `python train_model.py` whenever its CSV or training code changes.
3. Serve browser-only `dist` folders over localhost so JSON and scripts load normally.
4. Wait for the page's ready indicator before entering a prediction.
5. Keep the units printed beside each field and interpret the evidence panel together with the main label.
6. Never use the health demonstrations for diagnosis or urgent decisions.

Lab B

Glossary for a First-Time Reader

Word	Plain meaning in this report
Algorithm	A precise recipe of steps a computer follows.
Feature	An input fact, such as height, glucose, or one pixel value.
Target/label	The example answer supplied during supervised learning.
Training	Adjusting or constructing a model from examples; KNN mainly stores examples, and the current HMM page performs inference rather than training.
Inference/prediction	Applying an existing model to a new input.
Parameter	A learned or chosen number that affects behavior, such as a weight, threshold, K , or transition probability.
Epoch	One complete pass through all training examples.
Accuracy	Fraction of checked labels predicted correctly; its meaning depends completely on which rows were checked.
Overfitting	Learning training details so narrowly that new-data performance becomes worse.
Standardization	Subtracting a feature's mean and dividing by its standard deviation.
Confidence	A model score associated with a guess; it is not a guarantee.
Centroid	The coordinate-wise average position of a cluster.
Posterior	Probability of a hidden state after considering observations.
JSON	A text structure of names, arrays, numbers, and objects shared with the browser.
API	A defined request/response doorway between pieces of software.

Lab C

LaTeX Build and Error-Fixing Guide

C.1 Only required personal edit

At the top of this source, replace the bracketed placeholder and nothing else:

```
1 \newcommand{\StudentName}{Your Full Name}
```

The course code, ID, header, footer, title page, links, numbering, figures, and references are already configured.

C.2 Build commands

Run from the `report` directory so every relative asset path is correct:

```
1 tectonic CSE816_Machine_Learning_Lab_Report_20701003.tex
2 # Or, with a traditional TeX installation:
3 latexmk -pdf CSE816_Machine_Learning_Lab_Report_20701003.tex
```

Tectonic resolves the table of contents and cross-references in its compilation workflow. With manual `pdflatex`, compile at least twice.

C.3 Common errors and ready fixes

Table C.1: Common LaTeX problems and safe corrections.

Message/symptom	Correction
File <code>assets/...</code> not found	Start the build inside <code>report</code> , and confirm all seven PNG files remain in <code>report/assets</code> .
Undefined control sequence	Install the package named in the preamble or use the portable Tectonic compiler, which retrieves its bundle automatically.
Misplaced alignment tab <code>&</code>	In ordinary prose write <code>\&</code> ; raw <code>&</code> is reserved for table/formula alignment.
Missing <code>\$</code> inserted	Escape an underscore in ordinary text as <code>_</code> , or wrap a path in <code>\texttt{...}</code> with escaped underscores.
Percent sign removes text	Write <code>\%</code> when a printed percent symbol is intended.
Overfull box warning	The source already enables <code>breaklines</code> for code and <code>xurl</code> for links. For new long text, add a natural line break or shorten the unbreakable token.
References show <code>??</code>	Recompile; manual engines need two passes after changing headings, figures, or citations.
Header missing on chapter pages	Keep the redefined <code>plain</code> page style in the preamble; <code>\chapter</code> normally selects that style.

The critical page-style code is:

```

1 \pagestyle{fancy}
2 \fancyhf{}
3 \lhead{\textbf{\CourseCode}}
4 \rhead{\textbf{\StudentID}}
5 \cfoot{\thepage}
6 \fancypagestyle{plain}{% repeat the same header/footer for chapter pages
7   \fancyhf{}
8   \lhead{\textbf{\CourseCode}}
9   \rhead{\textbf{\StudentID}}
10  \cfoot{\thepage}
11 }
```

Bibliography

- [1] M. T. Hagan, H. B. Demuth, M. H. Beale, and O. De Jesús, *Neural Network Design*, 2nd ed. Stillwater, OK, USA, 2014. [Online]. Available: <https://hagan.okstate.edu/nnd.html>
- [2] F. Rosenblatt, “The perceptron: A probabilistic model for information storage and organization in the brain,” *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958, doi: 10.1037/h0042519.
- [3] T. Cover and P. Hart, “Nearest neighbor pattern classification,” *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, Jan. 1967, doi: 10.1109/TIT.1967.1053964.
- [4] Centers for Disease Control and Prevention, “Adult BMI calculator,” 2024. [Online]. Available: <https://www.cdc.gov/bmi/adult-calculator/>. [Accessed: Sep. 13, 2026].
- [5] American College of Cardiology, “2021 AHA/ACC chest pain guideline,” Oct. 2021. [Online]. Available: <https://www.acc.org/Guidelines/Guidelines/2021/10/28/12/34/Chest-Pain>. [Accessed: Sep. 13, 2026].
- [6] National Heart, Lung, and Blood Institute, “Heart attack diagnosis,” Mar. 2022. [Online]. Available: <https://www.nhlbi.nih.gov/health/heart-attack/diagnosis>. [Accessed: Sep. 13, 2026].
- [7] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*. Belmont, CA, USA: Wadsworth, 1984.
- [8] National Institute of Diabetes and Digestive and Kidney Diseases, “Diabetes tests and diagnosis,” Dec. 2022. [Online]. Available: <https://www.niddk.nih.gov/health-information/diabetes/overview/tests-diagnosis>. [Accessed: Sep. 13, 2026].
- [9] C. Cortes and V. Vapnik, “Support-vector networks,” *Machine Learning*, vol. 20, pp. 273–297, 1995, doi: 10.1007/BF00994018.
- [10] J. MacQueen, “Some methods for classification and analysis of multivariate observations,” in *Proc. 5th Berkeley Symp. Mathematical Statistics and Probability*, vol. 1, 1967, pp. 281–297. [Online]. Available: <https://digicoll.lib.berkeley.edu/record/113015?v=pdf>

- [11] L. R. Rabiner, “A tutorial on hidden Markov models and selected applications in speech recognition,” *Proceedings of the IEEE*, vol. 77, no. 2, pp. 257–286, Feb. 1989, doi: 10.1109/5.18626.
- [12] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition*, 2016, pp. 779–788. [Online]. Available: https://openaccess.thecvf.com/content_cvpr_2016/html/Redmon_You_Only_Look_CVPR_2016_paper.html
- [13] Ultralytics, “Ultralytics YOLOv8,” [Online]. Available: <https://docs.ultralytics.com/models/yolov8/>. [Accessed: Sep. 13, 2026].
- [14] T.-Y. Lin *et al.*, “Microsoft COCO: Common objects in context,” in *Computer Vision–ECCV 2014*, 2014, pp. 740–755, doi: 10.1007/978-3-319-10602-1_48.
- [15] Pallets, “Uploading files,” *Flask Documentation*. [Online]. Available: <https://flask.palletsprojects.com/en/stable/patterns/fileuploads/>. [Accessed: Sep. 13, 2026].
- [16] Pillow Contributors, “Pillow documentation,” [Online]. Available: <https://pillow.readthedocs.io/>. [Accessed: Sep. 13, 2026].
- [17] Ultralytics, “Train mode,” *Ultralytics YOLO Documentation*. [Online]. Available: <https://docs.ultralytics.com/modes/train/>. [Accessed: Sep. 13, 2026].